FP8 Block-Sparse Prefill Attention

Skip masked KV tiles, compute the kept ones in FP8

Tencent Hunyuan AI Infra · HPC-Ops

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Long prompts make prefill attention quadratic

- Prefill attends each token to every earlier one: $N^2/2$ pairs
- Long documents, code and agent traces push prompts to 64K–128K
- Block masks such as Stem keep only **25–31%** of causal blocks

A 128K prompt spends 1.5 s in dense attention

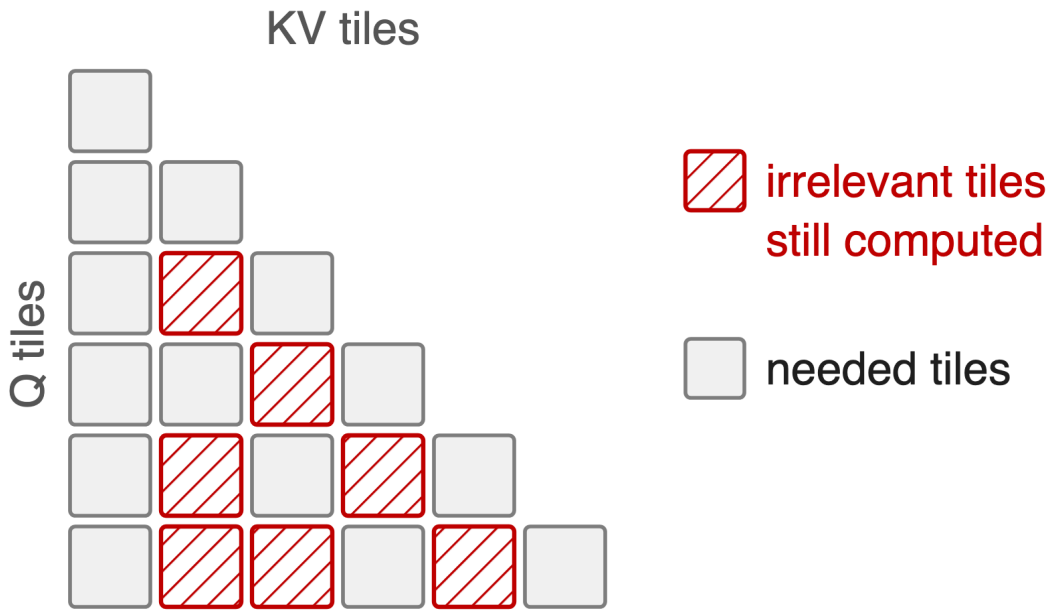
1540 ms

dense attention to prefill one 128K prompt (Llama-3.1-8B, H20); 24 ms at 16K, so 64× for 8× the length

Niu et al., "Stem", ICML 2026 (arXiv 2603.06274), Fig. 1 and §3.4: FlashAttention-2 BF16, batch 1

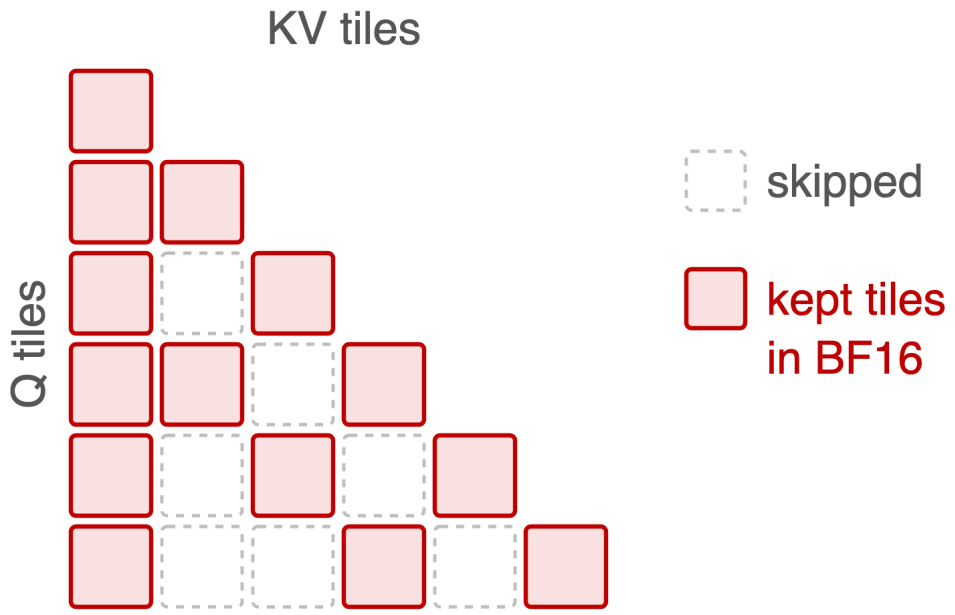
Today: compute every tile, or lose FP8

Dense FP8 (HPC-Dense, FA3)



every causal tile, at FP8 rate

BF16 block-sparse (MIT-BSA)



only kept tiles, at BF16 rate

Dense FP8 visits all causal 128 × 128 tiles; BF16 block-sparse kernels skip tiles but give up FP8 tensor cores

Insight: most tiles can go, if skipping is free

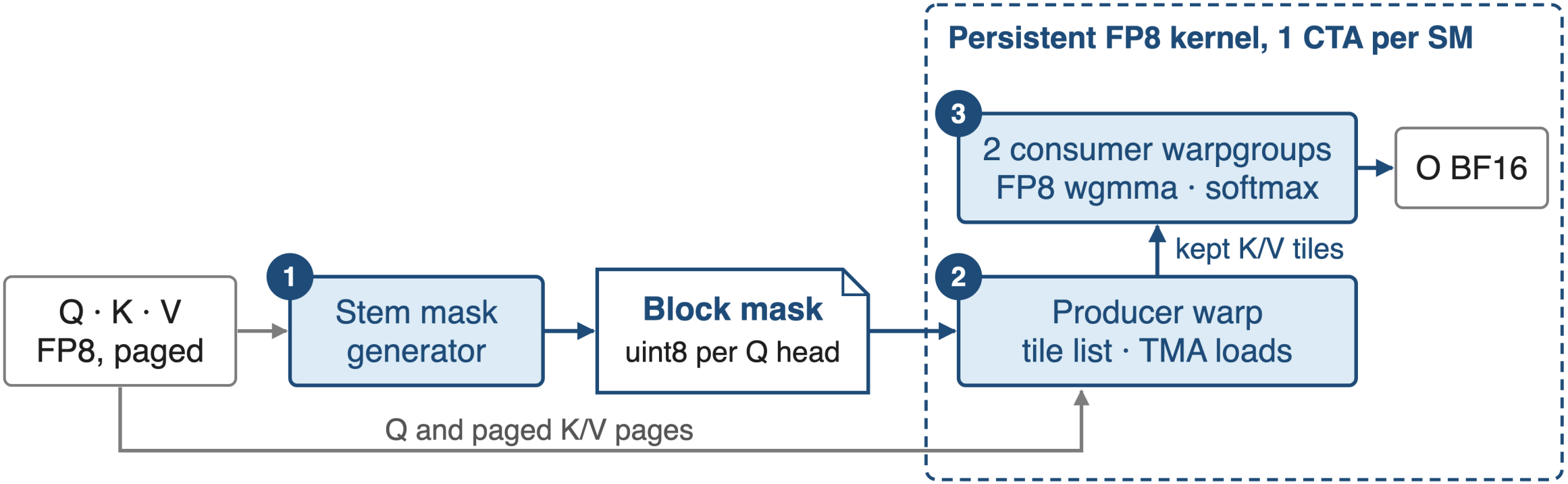
- Stem at 31% of blocks: LongBench **41.48** vs dense **42.02** (Llama-3.1-8B)
- A Q tile's mask row is known before its KV loop starts

Insight: most tiles can go, if skipping is free

- Stem at 31% of blocks: LongBench **41.48** vs dense **42.02** (Llama-3.1-8B)
- A Q tile's mask row is known before its KV loop starts

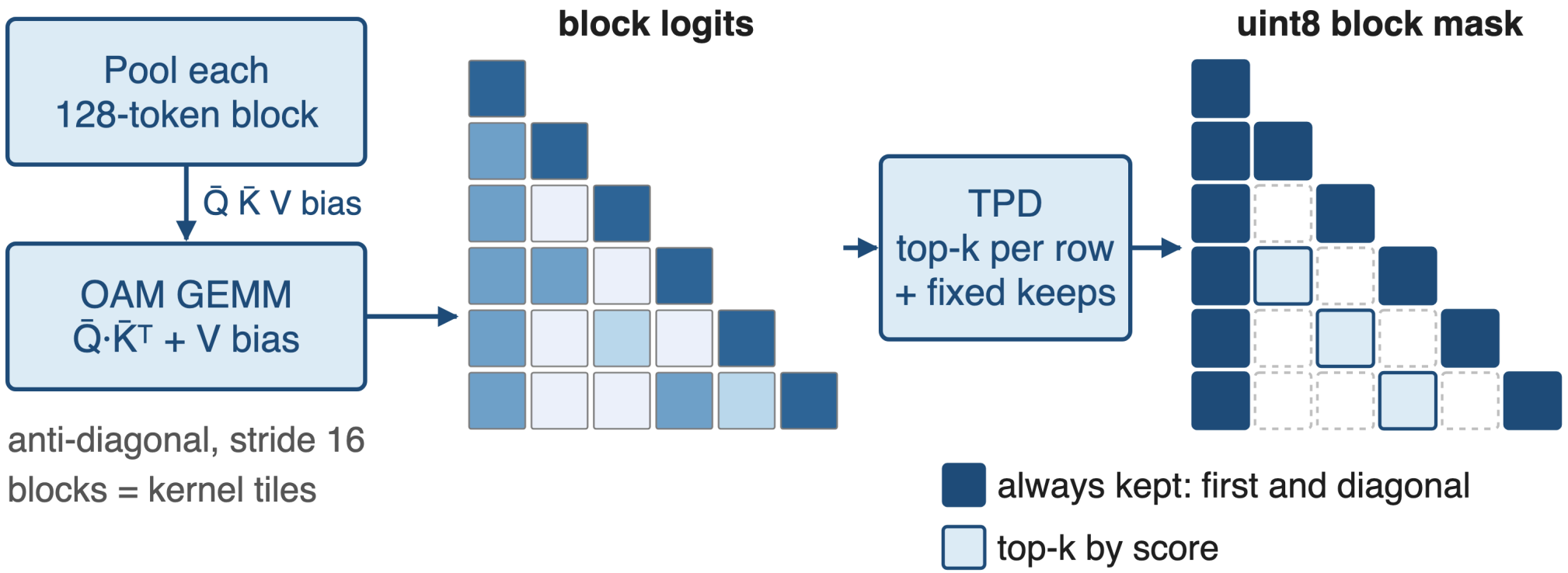
Turn the mask into a work list: skipped tiles cost no bytes and no FLOPs, kept tiles run in FP8

A block mask steers one persistent FP8 kernel



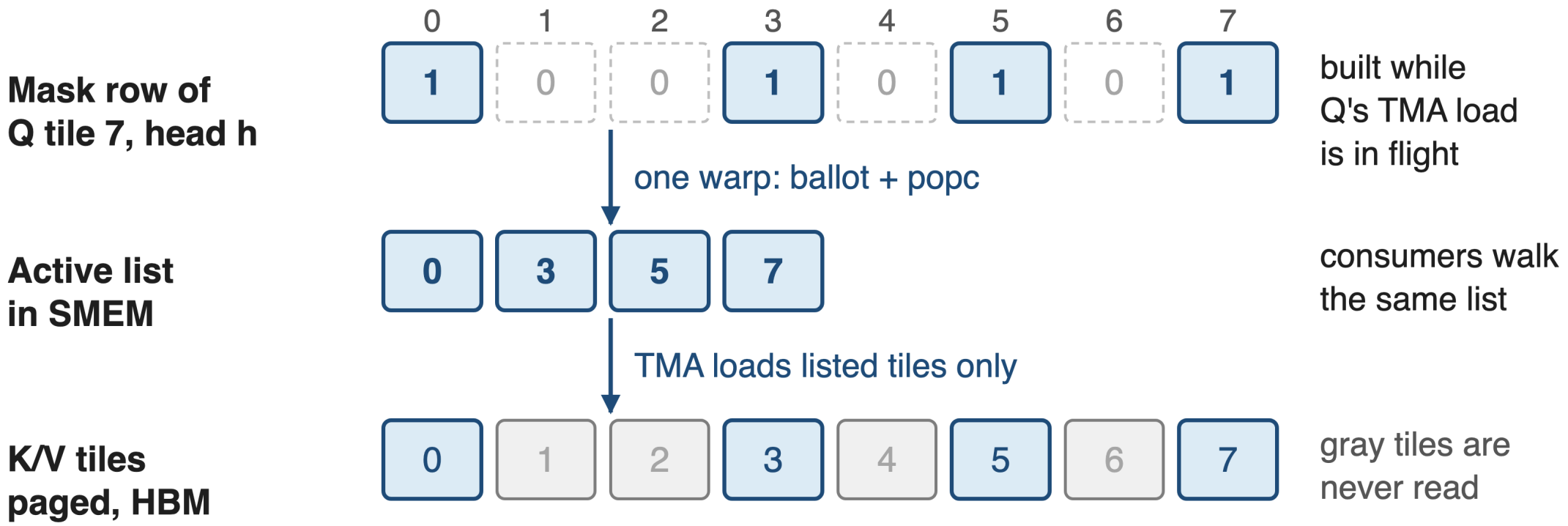
Mask = uint8 [batch, Q heads, Q tiles, KV tiles] over 128 × 128-token tiles; paged FP8 K/V cache

❶ Stem builds the mask on the GPU



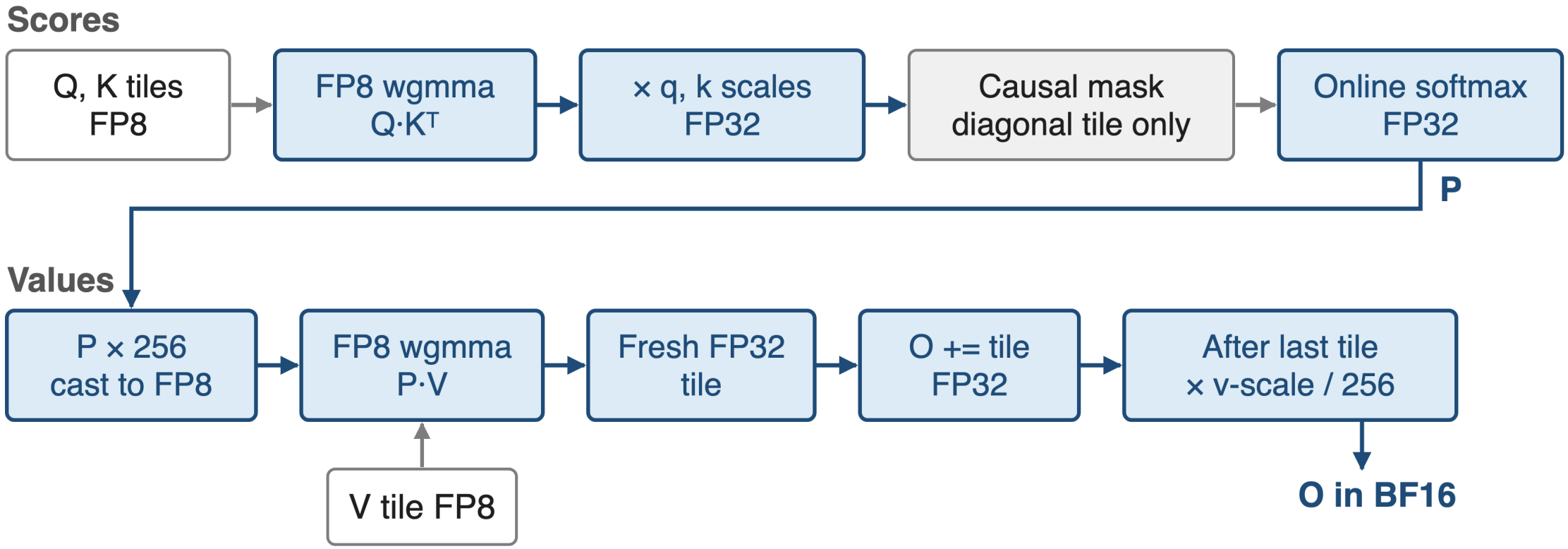
HPC-Ops defaults keep 4 initial and 4 window blocks; the top-k budget per row depends on prompt length

❷ The mask becomes a compact tile list



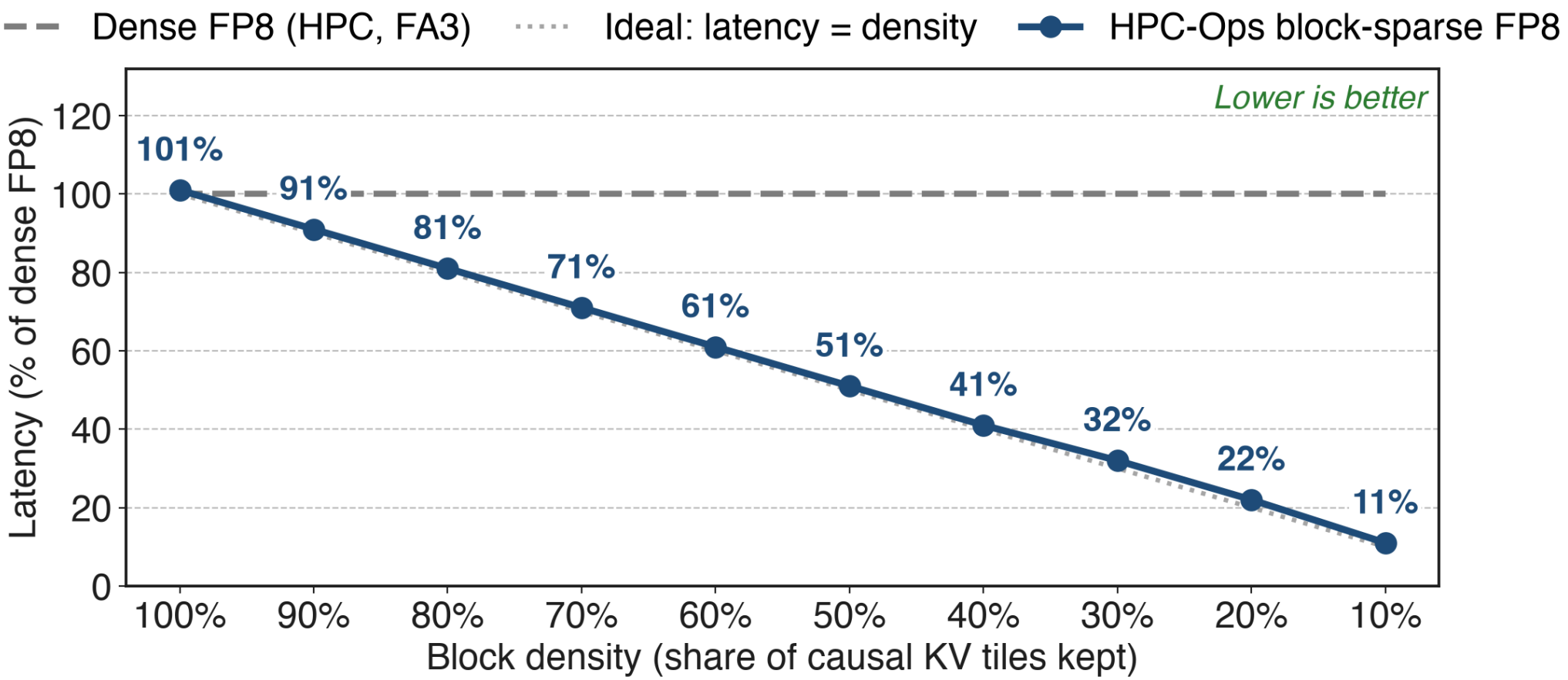
Producer warp: ballot and popcount compact the row while Q's TMA load is in flight; tiles beyond the mask are appended

❸ Kept tiles run end to end in FP8



Scales: Q per token and head; K per tensor or per token; V per tensor or per head; P × 256 undone in the epilogue

Latency tracks density: 11% at 10%



HPC-Ops PR #45 benchmark: batch 4, 64K tokens, 4 Q heads / 1 KV head, head_dim 128, H20

Up to 3.16× over BF16 sparse and dense FP8

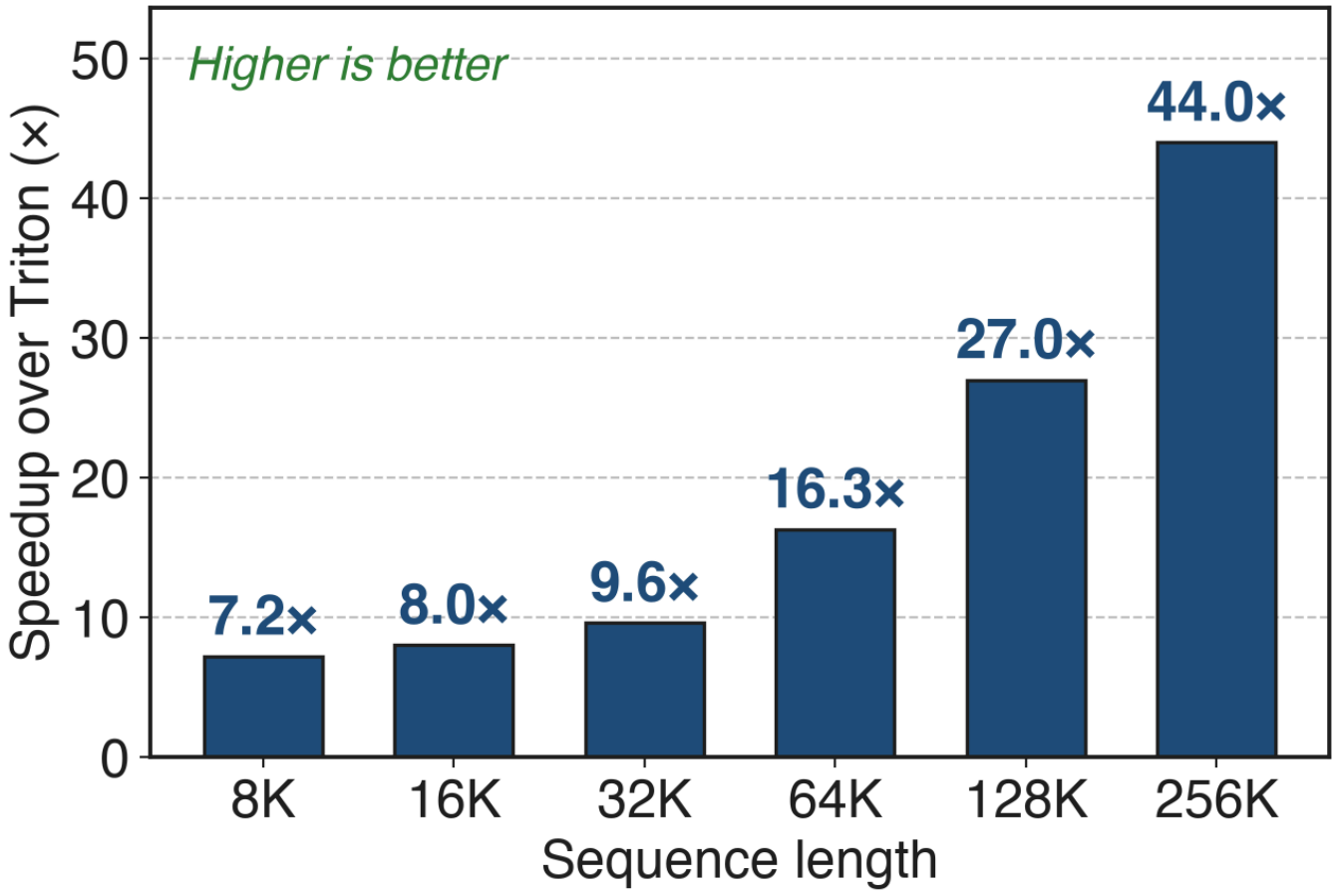
3.16×

maximum speedup across sparsity ratios and lengths vs MIT-BSA BF16, FlashPrefill-BSA BF16, HPC-Dense FP8 and FA3-Dense FP8

HPC-Ops README, Performance table, "Sparse Attention FP8" row (per-case table not published)

Stem masks build 7.2–44× faster than Triton

- **99.9%+** mask match vs FP32, from 14 tokens to 1M
- Gain grows with length: **7.2×** at 8K, **44×** at 256K



Costs: a good mask, SM90, head_dim 128

- **Accuracy = the mask's**: published accuracy used Stem with a BF16 kernel
- **Scope**: SM90, head_dim 128, page size dividing 128, prefill only
- **Edge case**: an all-skipped Q tile gives NaN; keep the diagonal block

Summary

- Dense prefill attention: **1540 ms** at 128K on H20, quadratic in length
- **Insight**: mask → tile list, so skipped tiles are free and kept tiles run in FP8
- Latency **11%** of dense at 10% density; up to **3.16×**; masks **7.2–44×** faster
- Library: deck 26 (HPC-Ops); decode side: deck 07

https://github.com/Tencent/hpc-ops